

毕业设计（论文）

题目：基于 AI Agent 的高性能分布式通讯系统

学生姓名：

学 号：

班 级：

专 业：

指导教师：

2026 年 5 月

基于 AI Agent 的高性能分布式通讯系统

摘要

随着大语言模型与工具调用技术的发展，AI Agent 已从单轮问答工具逐步演进为能够理解目标、分解任务、调用外部工具并持续反馈的自治执行单元。多个 Agent 共同工作时，系统不再只面对模型能力问题，还必须解决任务编排、跨进程通信、状态一致性、执行隔离、实时可观测性和故障恢复等工程问题。传统多智能体框架多强调角色协作与提示词流程，而在本地化、高性能、可审计的分布式通信层面仍存在进一步优化空间。

本文围绕“基于 AI Agent 的高性能分布式通讯系统”展开研究与设计，实现了一套名为 Fuxi 的 Rust 平台。系统采用“玄女—门客”的分层协作模式，将面向用户的顶层 Agent 与执行任务的工作 Agent 解耦；以 A2A 语义描述 Agent 间任务、消息和能力发现；以 tokio broadcast 与 SQLite WAL 构建实时事件总线；以 Firehose、WebSocket、SSE 和 IM API 提供多端观测能力；以 Git worktree 和分层 sandbox 保证任务执行隔离。系统同时提供长期记忆、定时触发、角色加载、分布式任务入队和跨节点事件回传等支撑模块。

测试结果表明，Fuxi 在 8 个 worker、10ms 模拟任务条件下达到 663.90 tasks/s 的吞吐量；在 500 次样本测试中，任务派发 p50 延迟为 32.78ms、p99 延迟为 35.81ms，跨节点事件流 p50 延迟为 0.09ms、p99 延迟为 0.25ms。上述结果说明，事件驱动与追加式日志结合的通信架构能够在本地优先的 AI Agent 协作场景中兼顾吞吐、实时性与可追溯性。

关键词：AI Agent；分布式通信；多智能体系统；事件总线；A2A 协议；Rust

ABSTRACT

With the development of large language models and tool-use techniques, AI agents are evolving from single-turn assistants into autonomous execution units capable of goal understanding, task decomposition, external tool invocation and continuous feedback. When multiple agents collaborate, a system must address not only model capability, but also task orchestration, inter-process communication, state consistency, execution isolation, real-time observability and fault recovery.

This thesis designs and implements Fuxi, a high-performance distributed communication system based on AI agents. Fuxi adopts a hierarchical collaboration pattern that separates a user-facing orchestrator agent from worker agents. It uses A2A-style task and message semantics, builds a real-time event bus with tokio broadcast and SQLite WAL, exposes observability through Firehose, WebSocket, SSE and IM APIs, and isolates execution through Git worktrees and layered sandboxes. Experimental results show that Fuxi reaches 663.90 tasks/s with 8 workers under 10ms simulated jobs, while the p50 and p99 latencies of cross-node event flow are 0.09ms and 0.25ms respectively.

Key words: AI Agent; Distributed Communication; Multi-Agent System; Event Bus; A2A Protocol; Rust

目 录

绪论	6
1.1 研究背景及意义	6
1.2 国内外研究现状	7
1.3 论文的研究内容	10
1.4 论文结构安排	10
系统总体设计	10
2.1 系统目标及主要任务	10
2.2 需求分析	11
2.3 系统整体架构	11
2.4 通信模型与性能指标	12
系统核心模块设计	13
3.1 模块划分	13
3.2 Agent 通信协议模块	14
3.3 事件总线模块	14
3.4 编排与任务调度模块	15
3.5 执行适配器与工作区隔离模块	16
3.6 观测与 IM 接入模块	16
3.7 记忆、角色与触发器模块	16
系统实现	17
4.1 开发环境与技术选型	17
4.2 Rust Workspace 结构	17
4.3 事件数据结构实现	18
4.4 任务派发流程实现	18
4.5 分布式通信实现	19
4.6 关键流程伪代码	19
系统测试与结果分析	20
5.1 测试目标与测试方法	20
5.2 测试用例覆盖	20

5.3 吞吐量测试	20
5.4 延迟测试	21
5.5 功能验收分析	21
总结与展望	23
6.1 总结	23
6.2 不足与展望	24
参考文献	24
致谢	26

绪论

1.1 研究背景及意义

AI Agent 是多智能体系统在大语言模型时代的新形态。传统多智能体研究强调自治实体之间的感知、通信、协商与协同，Wooldridge 对 Multi-Agent Systems 的定义为后续 Agent 架构提供了理论基础[1]。大语言模型出现后，Agent 获得了以自然语言理解任务、以工具调用改变环境、以记忆模块保留经验的能力，相关综述将规划、记忆、工具使用和行动反馈视为 LLM-based autonomous agents 的核心组成[2]。因此，AI Agent 系统的研究对象已经从“单模型推理效果”扩展为“模型、工具、协议、运行时和观测系统共同构成的工程平台”。

在复杂软件开发、数据分析和自动化运维等场景中，单个 Agent 往往难以同时承担需求澄清、计划拆解、代码实现、测试验证和结果交付等职责。CAMEL 通过角色扮演方式研究通信 Agent 的协作能力[3]，AutoGen 将多 Agent 对话抽象为可编程的应用框架[4]，MetaGPT 将软件工程中的标准作业流程编码为多角色协作机制[5]，ChatDev 则进一步验证了“由多个通信 Agent 组成虚拟软件公司”的可行性[6]。这些研究说明，多 Agent 协作能够提高任务分解能力和过程可解释性，但也暴露出通信协议不统一、任务生命周期不清晰、状态难以追踪等工程问题。

对于本地化运行的 AI Agent 平台而言，高性能分布式通信具有直接意义。第一，Agent 执行过程通常包含大量工具调用、文件读写、测试命令和中间消息，系统需要用低延迟通道将这些状态及时反馈给用户。第二，多个 Agent 并发执行时，任务调度与消息回传必须避免轮询造成的延迟和资源浪费。第三，AI Agent 的输出具有概率性，工程系统需要通过事件日志、会话恢复和执行隔

离降低不可追溯风险。AIOS 从操作系统视角讨论了 LLM Agent 的资源调度、上下文管理和访问控制问题[7]；SWE-agent 和 OpenHands 则表明，面向 Agent 设计的计算机接口和沙箱环境会显著影响软件工程任务表现[8][9]。由此可见，围绕 AI Agent 构建高性能通信底座，是使多 Agent 协作从实验演示走向可用系统的关键。

1.2 国内外研究现状

从 Agent 行为模式看，ReAct 提出将推理轨迹与外部行动交替生成，使模型能够在推理过程中调用环境反馈修正计划[10]；Toolformer 探索模型自动学习何时调用 API、如何传入参数以及如何吸收结果[11]；MRKL 将大语言模型与外部知识源、符号推理模块组合，为“模型作为协调者，工具作为执行器”的系统结构提供了早期思路[12]。这些工作共同推动了 Agent 从纯语言生成向“推理—行动—观察”闭环演进。

从多 Agent 协作框架看，AutoGen、MetaGPT、CAMEL、ChatDev 等系统主要围绕角色、对话和流程进行设计[3-6]。它们证明了角色分工和任务链路的有效性，但多数系统以 Python 框架或云端应用为主要形态，通信层通常服务于上层协作逻辑，而非作为可独立审计、可回放、可跨节点扩展的底层平台来设计。近期关于 AI Agent 协议的综述指出，Agent 通信协议正从工具接入协议向 Agent-to-Agent 协议扩展，协议的互操作性、安全边界和任务语义成为核心问题[13]。关于任务编排协议的经验比较也表明，不同协议在任务委派、工具调用和可观测性方面存在明显差异[14]。

从通信协议和分布式系统基础看，A2A 协议试图用 Agent Card、Task、Message、Artifact 等概念描述独立 Agent 之间的互操作关系[15]；MCP 则强调

模型与外部工具、数据源之间的标准化连接[16]。面向 LLM Agent 通信安全的综述进一步指出，Agent 协议不仅要解决消息格式问题，还要处理身份认证、权限隔离、提示注入和跨边界调用风险[26]；面向企业级多 Agent 编排的研究则将协议、治理、可观测性和审计能力视为规模化落地的共同基础[27]。在更传统的分布式系统领域，MapReduce 证明了将大规模任务分解、调度和容错封装到运行时中的价值[17]，GFS 强调了廉价硬件上高吞吐、容错存储的系统设计思路[18]，Kafka 以追加式日志为核心实现高吞吐消息处理[19]，Raft 通过更易理解的一致性协议降低了分布式状态复制的实现复杂度[20]。这些工作虽然并非面向 AI Agent，但其关于任务调度、日志追加、故障恢复和状态一致性的思想，对 Agent 通信系统具有重要借鉴意义。

在工程实现层面，SQLite WAL 将事务写入追加式日志，在并发读写和崩溃恢复之间取得平衡[21]；Tokio 的异步 channel 提供了 Rust 高并发系统中常用的消息传递基础，其中 broadcast 支持多生产者、多消费者的发布订阅模型[22]；Axum 提供 WebSocket 与 SSE 等网络接口能力[23]；Git worktree 支持同一仓库下多个工作树并存，为 Agent 任务隔离提供了轻量级方案[24]。结合上述研究和工具，可以构建一种本地优先、事件驱动、可观测的 AI Agent 分布式通信平台。

1.3 论文的研究内容

本文设计并实现的 Fuxi 系统主要面向 AI Agent 多任务并行和分布式通信场景，重点研究内容包括：第一，建立面向 AI Agent 的通信抽象，将用户意图、任务创建、任务派发、Agent 响应、工具调用和结果交付统一表示为可持久化事件；第二，设计“玄女—门客”的分层协作模式，使顶层 Agent 负责用户交互、

任务判断和结果汇报，工作 Agent 负责具体执行；第三，构建基于 tokio broadcast 和 SQLite WAL 的事件总线，实现实时推送与历史回放并存；第四，设计工作区隔离、角色加载、长期记忆、定时触发和 IM 接入等支撑模块；第五，通过单元测试、集成测试、端到端验收和吞吐/延迟基准测试验证系统可用性。

根据当前代码实现，Fuxi 正文重点体现的模块包括：通信协议模块、事件总线模块、Agent 编排模块、分布式任务调度模块、执行适配器模块、工作区隔离模块、实时观测模块、记忆与经验模块、定时触发模块、交付物模块、IM/PWA 接入模块和测试评估模块。这些模块共同服务于一个目标：在多 Agent 并发工作时，以低延迟、可追溯、可恢复的方式完成任务通信与状态流转。

1.4 论文结构安排

第一章，绪论。阐述 AI Agent 分布式通信系统的研究背景、国内外研究现状、研究内容和论文结构。

第二章，系统总体设计。分析系统需求，给出整体架构、通信模型、性能指标和关键公式。

第三章，系统核心模块设计。分别说明通信协议、事件总线、编排调度、工作区隔离、记忆、触发器和观测模块。

第四章，系统实现。结合 Fuxi 代码仓库说明 Rust workspace、数据结构、任务流程和接口实现。

第五章，系统测试与结果分析。介绍测试环境、测试用例、吞吐量、延迟指标和结果分析。

第六章，总结与展望。总结本文工作并讨论后续优化方向。

系统总体设计

2.1 系统目标及主要任务

Fuxi 的目标不是将多个 Agent 简单串联，而是为多 Agent 协作提供一套稳定的通信与运行时基础设施。系统面向的典型场景是：用户向顶层 Agent 提出较复杂的软件工程或运维需求，顶层 Agent 将需求分解为若干任务，选择合适角色的工作 Agent 执行，并在执行过程中持续接收状态事件，最终向用户汇报结果。该过程要求系统既能支持 Agent 的自然语言协作，也能在工程层面提供严格的事件记录、任务状态和故障边界。

系统主要任务包括：（1）实现 Agent 与 Agent 之间的标准化任务通信；（2）实现用户、玄女、门客、IM 端和观测端之间的实时事件传播；（3）实现跨进程和跨节点任务派发；（4）实现工作区隔离与交付物管理；（5）实现长期记忆、角色技能和定时触发等扩展能力；（6）通过完整测试体系保证系统行为可验证。

2.2 需求分析

功能需求方面，系统需要支持 Agent 注册、任务创建、任务派发、消息发送、事件发布、事件订阅、历史回放、工作区创建、任务恢复、角色加载、记忆查询、定时触发和 IM 接入。非功能需求方面，系统需要满足低延迟、高吞吐、可恢复、可观测、可扩展和本地优先等要求。其中低延迟要求事件从产生到被观测端接收的时间尽可能短；高吞吐要求多个 worker 并发执行时调度损耗可控；可恢复要求系统崩溃或 Agent 死亡后仍能通过事件日志恢复关键上下文；可观测要求用户能够实时看到任务进展和异常状态。

2.3 系统整体架构

Fuxi 采用分层架构。最底层为 fuxi-core，负责定义 Agent、Task、Workspace、Event 等核心类型。通信层由 fuxi-a2a 和 fuxi-events 构成，前者描述 Agent 间任务与消息语义，后者负责事件发布、订阅和持久化。编排层由 fuxi-orchestrator 实现，负责维护 Agent shelf、任务派发、系统事件桥接和分布式任务入队。执行层由 fuxi-agent-cc、fuxi-agent-codex 和 fuxi-workspace 构成，负责启动外部 CLI Agent 并为其分配隔离工作树。观测层由 fuxi-firehose、fuxi-im 和 TUI 构成，负责将事件流以 WebSocket、SSE、REST、PWA 或终端界面形式呈现给用户。

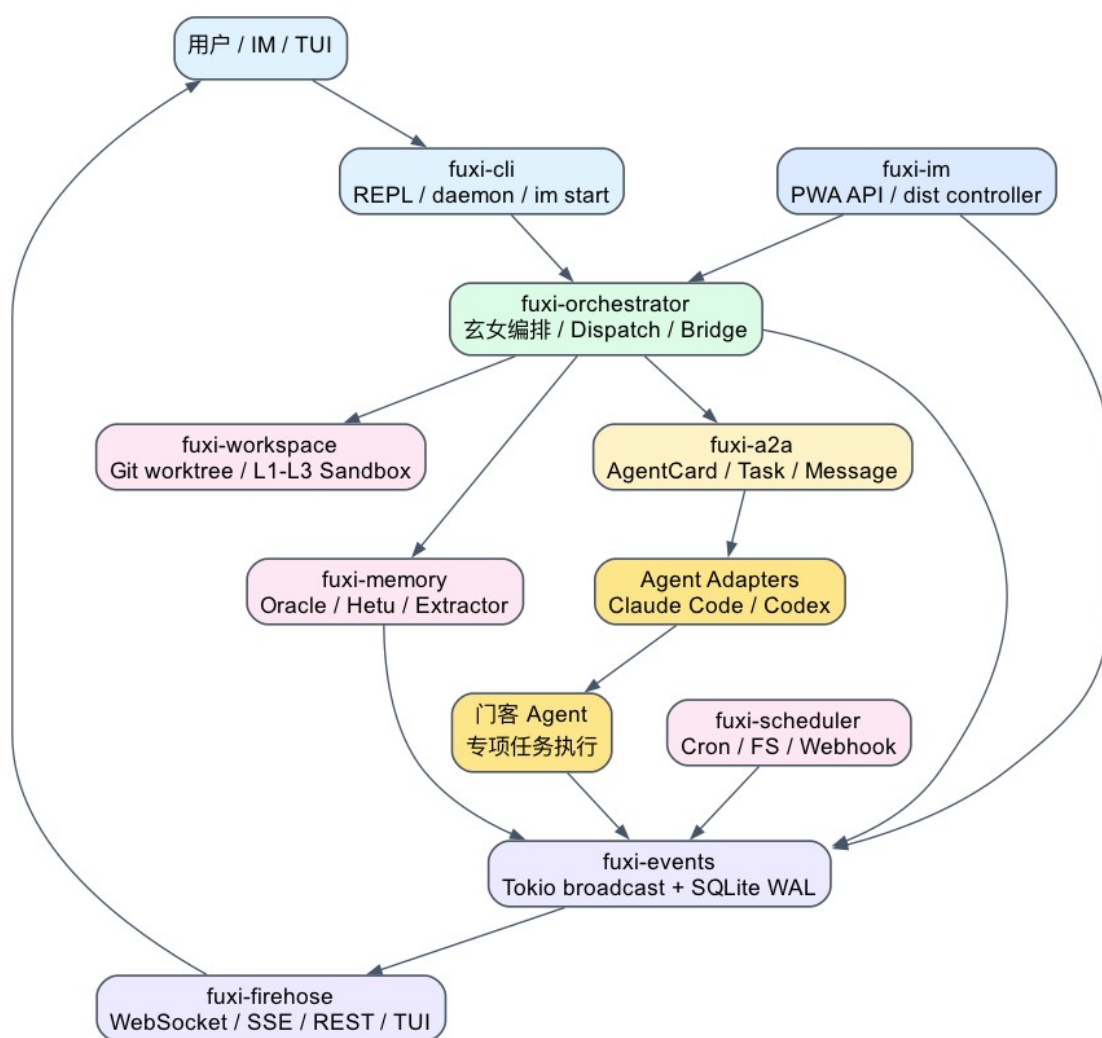


图 2.1 Fuxi 系统总体架构图

2.4 通信模型与性能指标

Fuxi 的通信模型以事件为中心。一次用户请求首先被表示为 UserPrompted，再由编排层生成 TaskCreated 事件；本地任务分配给某个 Agent 后产生 TaskDispatched 事件；Agent 执行过程中产生 ToolCallStarted、ToolCallFinished、AgentResponded 等事件；任务结束时统一产生 TaskStateChanged 事件并进入 Done 或 Cancelled 等终态。分布式任务在入队前由 home 进程补发 TaskCreated，远端 worker 执行过程中的 Agent 事件再通过 dist event 路径回流。所有事件一方面通过 broadcast 实时推送给订阅者，另一方面写入 SQLite WAL 支撑回放。

$$L_{e2e} = L_{enqueue} + L_{dispatch} + L_{execute} + L_{event} \quad (2-1)$$

式 (2-1) 中， L_{e2e} 表示端到端任务耗时， $L_{enqueue}$ 表示任务入队开销， $L_{dispatch}$ 表示调度开销， $L_{execute}$ 表示 Agent 实际执行耗时， L_{event} 表示事件传播开销。对于 AI Agent 系统而言，模型推理和工具执行常常占据主要时间，但当任务粒度变细、worker 数量增加时，通信层开销会显著影响交互体验。

$$\text{Throughput} = N / T \quad (2-2)$$

$$\text{Overhead} = 1 - \text{Throughput} / \text{Throughput_max} \quad (2-3)$$

式 (2-2) 中， N 表示完成任务数， T 表示总耗时；式 (2-3) 中， Throughput_max 表示理想吞吐上限。本文在第五章使用该指标分析 Fuxi 在不同 worker 数量和任务粒度下的性能损耗。

2.5 本章小结

本章从目标、需求、架构和性能指标四个方面给出了 Fuxi 的总体设计。系统以事件驱动作为核心通信方式，将 Agent 协作问题转化为任务生命周期与事件流管理问题，为后续模块设计奠定基础。

系统核心模块设计

3.1 模块划分

Fuxi 的模块划分遵循“核心类型稳定、通信层独立、编排层集中、执行层可替换、观测层多出口”的原则。系统模块关系如图 3.1 所示。

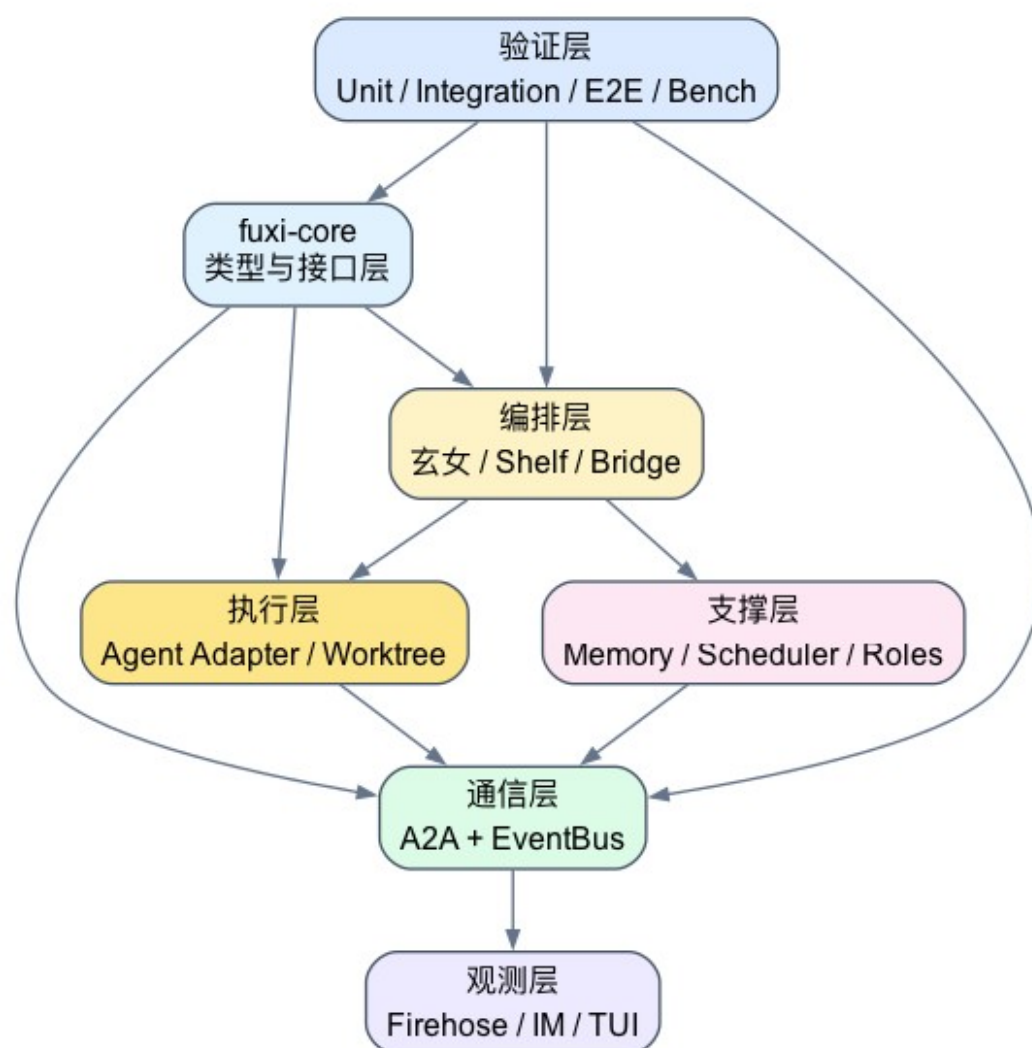


图 3.1 系统模块关系图

3.2 Agent 通信协议模块

通信协议模块主要由 fuxi-a2a 提供。A2A 协议强调独立 Agent 之间的能力描述、任务委派和消息交互[15]，而 Fuxi 在实现中保留 AgentCard、Task、Message 等核心语义，并结合本地执行特点进行了轻量化处理。与 MCP 更偏向模型连接外部工具不同[16]，Fuxi 更关注 Agent 与 Agent 之间的任务流转，即谁创建任务、谁接收任务、谁报告状态、谁交付结果。需要说明的是，Fuxi 的 A2A 消息语义与事件总线并非同一个层次：A2A 描述 Agent 间任务和消息的 wire 结构，EventKind 则描述系统内部“发生了什么”的事实词汇。

协议模块的设计重点是保持 Agent 适配器的可替换性。Claude Code、Codex 或未来其他 CLI Agent 都可以被包装成统一 Agent trait，只要它能够接收任务、返回消息、报告状态即可。这样，Fuxi 不需要 fork 或绑定某个特定 CLI，而是把外部 Agent 当作可管理的操作系统进程。

3.3 事件总线模块

事件总线由 fuxi-events 实现，是系统高性能通信的核心。Tokio broadcast channel 的特点是一个发送者发布消息后，多个接收者均可独立接收，适合 Firehose、IM、TUI、系统桥接等多个观察者同时订阅[22]。但单纯内存 channel 无法解决历史回放和崩溃恢复问题，因此 Fuxi 将事件同时追加写入 SQLite 数据库，并启用 WAL 模式。SQLite WAL 的提交通过追加日志完成，读者可以在写入同时继续读取旧版本数据[21]，这与事件溯源模型高度契合。

事件总线设计兼顾实时性与可靠性。实时路径中，事件通过 broadcast 直接抵达订阅者；持久路径中，事件以 append-only 形式写入 SQLite；恢复路径

中，订阅者可以从某一事件序号之后 replay。Lamport 在分布式系统事件排序研究中指出，事件的先后关系是理解分布式行为的基础[28]。Fuxi 通过事件元信息记录 agent、task、timestamp 和 kind，使调试、审计和 UI 渲染都基于同一事实源。

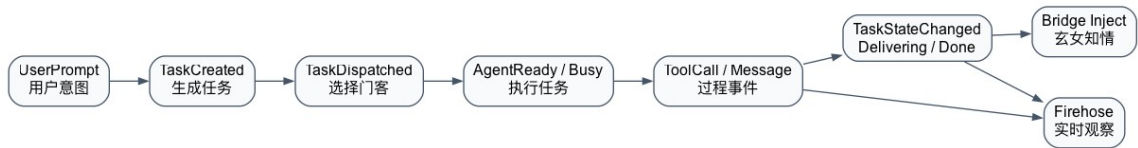


图 3.2 任务生命周期与事件流转图

3.4 编排与任务调度模块

编排模块由 fuxi-orchestrator 实现，其核心职责是维护系统中所有 Agent 的状态，并根据任务需求选择合适 Agent。Fuxi 将直接面向用户的顶层 Agent 称为“玄女”，将负责执行具体任务的 worker Agent 称为“门客”。这种设计与 MetaGPT、ChatDev 中的角色分工思想一致[5][6]，但 Fuxi 更强调执行层可观测和任务事件可追踪。

任务调度分为本地调度和分布式调度。本地调度直接从 shelf 中选择 idle Agent 并派发任务；分布式调度在任务包含 pinned_node 或 required_tags 时进入 dist 队列，由目标节点执行。该设计借鉴了 MapReduce 将任务分解和调度封装在运行时中的思想[17]，但 Fuxi 的任务粒度更偏交互式 Agent 工作单元，而不是大数据批处理。

3.5 执行适配器与工作区隔离模块

执行适配器负责将外部 CLI Agent 包装成统一接口。fuxi-agent-cc 适配 Claude Code，fuxi-agent-codex 适配 Codex CLI。Codex adapter 采用 lazy spawn 方式，一次 dispatch fork 一次 codex exec，使短任务执行和资源释放更直接。工

作区隔离由 fuxi-workspace 提供，利用 Git worktree 在同一仓库下创建多个独立工作树[24]，从而避免多个 Agent 修改同一工作目录造成冲突。

Fuxi 将工作区划分为 L1 read-only、L2 ephemeral 和 L3 persistent sandbox。L1 用于只读分析，L2 用于临时任务，L3 用于需要保留交付物和上下文的长期任务。该设计与 SWE-agent、OpenHands 对 Agent-Computer Interface 和沙箱执行环境的重视一致[8][9]，也回应了 AIOS 关于资源隔离和访问控制的要求[7]。

3.6 观测与 IM 接入模块

观测模块由 fuxi-firehose 与 fuxi-im 构成。Firehose 将事件流以 TUI、WebSocket、SSE 和 REST 四种形式输出；IM 模块提供 PWA 后端、节点视图、任务视图、通知与上传能力。Axum 的 SSE 与 WebSocket 能力为 Rust 网络服务提供了基础支持[23]。通过观测模块，用户无需轮询即可看到 Agent 上线、任务接单、工具调用、任务完成和异常死亡等状态。

3.7 记忆、角色与触发器模块

记忆模块 fuxi-memory 包含 oracle_facts、user_profile 和 hetu_patterns 等表，用于存储事实、用户画像和可复用经验。Voyager 的 skill library 表明，Agent 的长期能力可以通过可检索、可组合的经验库持续积累[25]；Fuxi 将该思想转化为本地 SQLite 存储和显式查询接口。角色模块 fuxi-skills 负责加载 ROLE.md 和相关指令，使不同 Agent 具备稳定职责边界。触发器模块 fuxi-scheduler 支持 cron、once、fs-watch 和 webhook，使系统能够在时间或外部事件到达时主动唤醒玄女。

3.8 本章小结

本章对 Fuxi 的核心模块进行了设计说明。系统以通信协议和事件总线为中心，通过编排层组织 Agent，通过执行层隔离任务，通过观测层暴露状态，通过记忆、角色和触发器支撑长期运行。

系统实现

4.1 开发环境与技术选型

Fuxi 采用 Rust 语言实现，原因在于 Rust 兼具高性能、内存安全和成熟异步生态，适合构建长时间运行的本地服务与并发通信系统。异步运行时选择 Tokio，Web 服务使用 Axum，持久化存储使用 SQLite，前端 IM 部分采用 PWA 形态。系统以 Cargo workspace 组织多个 crate，使通信、编排、执行、存储和界面模块相互解耦。

4.2 Rust Workspace 结构

Crate	主要职责	所属层次
fuxi-core	核心 trait、Task、Event、Workspace 类型	全局基础类型
fuxi-events	EventBus、SQLite WAL、事件回放	通信核心
fuxi-a2a	A2A wire、JSON-RPC、server/client	Agent 间协议
fuxi-orchestrator	Fuxi、Shelf、Bridge、dispatch	任务编排
fuxi-agent-cc/codex	外部 CLI Agent 适配器	执行适配
fuxi-workspace	Git worktree、sandbox、deliverables	执行隔离
fuxi-firehose	TUI、WebSocket、SSE、REST	实时观测
fuxi-memory	Oracle、Hetu、Extractor	长期记忆
fuxi-scheduler	Cron、fs、webhook trigger	主动触发
fuxi-im	IM API、PWA、dist controller	移动端与分布式入口

表 4.1 Fuxi workspace 主要 crate 与职责

4.3 事件数据结构实现

事件结构由 EventMeta 与 EventKind 组成。EventMeta 记录事件 ID、时间戳、Agent ID、Task ID、source_node_id 等元信息；EventKind 使用 Rust enum 表示具体事件类型，如 AgentReady、TaskCreated、TaskDispatched、TaskStateChanged、AgentResponded、ToolCallStarted、TriggerFired、AgentDead、WorkerRegistered、WorkspaceCreated、DeliverableProduced 等。使用 enum 的好处是编译器能够在 match 时检查分支覆盖，减少事件新增后 UI 或持久化逻辑遗漏的风险。

事件写入流程分为三个步骤：首先，业务模块构造 Event；其次，EventBus 将事件发送到 broadcast channel；最后，EventStore 将事件序列化写入 SQLite。若某个订阅者处理速度落后，broadcast 的 lag 检测能够提示订阅端发生丢帧风险[22]；而持久化日志仍可用于后续补读。

4.4 任务派发流程实现

当用户输入任务后，玄女判断任务类型并调用编排层。编排层创建 Task，写入 TaskCreated 事件，然后根据 role、required_tags、pinned_node 和 project_id 等条件选择 worker。如果任务显式指定 pinned_node，或 required_tags 非空，系统通过 dist enqueue 进入分布式队列；如果任务关联 project_id 且未显式 pin，编排层会根据项目 host_nodes 与节点负载自动选择最闲节点并写入 pinned_node；如果上述条件均不满足，则走本地 worker 路径。任务执行期间，Agent adapter 从外部 CLI 的输出流中解析消息和工具调用，再转为标准事件写入总线。

为了避免顶层 Agent 通过轮询方式查询 worker 状态，Fuxi 引入 SystemEventBridge。该桥接器订阅 AgentDead、TriggerFired、AgentRequestReview、TaskDone 等关键系统事件，并将其转换为注入玄女上下文的系统提示。这样，玄女拥有知情权，但不需要主动轮询，从而符合“真实时，不轮询”的系统原则。

4.5 分布式通信实现

分布式通信主要服务于多节点 worker 场景。IM 启动时内嵌 dist controller，/api/* 使用 IM cookie auth，/dist/* 使用 HMAC auth。home 节点会自注册，并启动 embedded dist worker 消费 pinned_node=home 的任务；其他节点通过 register 和 heartbeat 上报 tags、max_concurrency 与 inflight 状态。任务拉取时，worker 必须同时满足三个条件：未被 pin 或 pin 到本节点、required_tags 是 worker.tags 的子集、当前并发未超过 max_concurrency。远端 worker 完成任务后，将事件回传到 home 节点的 EventBus，使用户在同一 Firehose 中观察本地与远端任务。

该实现没有采用强一致复制，而是将 Fuxi 当前阶段的分布式范围限定为任务队列、节点心跳、inflight 回收与事件回传。这一选择降低了实现复杂度，也符合系统规模：Fuxi 面向个人或小团队的本地优先 Agent 协作，而非大规模数据中心服务。对于需要更高一致性的场景，后续可以借鉴 Raft 等一致性算法 [20]。

4.6 关键流程伪代码

系统任务派发伪代码如下：

```
Input: user_prompt  
task = create_task(user_prompt)
```

```
publish(TaskCreated(task))
worker = select_worker(task.role, task.tags, task.pinned_node)
if worker is remote:
    enqueue_dist_task(task, worker.node)
else:
    publish(TaskDispatched(task, worker))
    worker.dispatch(task)
while task not finished:
    event = read_agent_stream(worker)
    publish(event)
publish(TaskStateChanged(task, Done))
```

4.7 本章小结

本章说明了 Fuxi 的工程实现方式。系统通过 Rust workspace 保持模块边界，通过 EventKind 统一事件词汇，通过 Agent adapter 兼容不同 CLI Agent，通过 SystemEventBridge 避免轮询，通过 dist controller 支持跨节点任务执行。

系统测试与结果分析

5.1 测试目标与测试方法

系统测试目标包括正确性、实时性、吞吐能力、容错能力和用户验收。正确性主要通过 Rust 单元测试和集成测试验证；实时性和吞吐能力通过 benchmark 脚本测量；容错能力通过 gateway restart、chaos resilience、AgentDead 等测试覆盖；用户验收通过 TUI 修复批次和 M2 验收文档记录。

5.2 测试用例覆盖

模块	测试文件	测试目标
fuxi-a2a	roundtrip	验证 A2A wire 编解码与往返一致性
fuxi-events	integration	验证事件发布、订阅和回放
fuxi-agent-cc/codex	fixture_stream / real smoke	验证外部 CLI 输出解析与真实启动
fuxi-cli	chaos_resilience / gateway_restart / im_dist_layer	验证 CLI、网关重启和分布

		式层
fuxi-im	router_smoke / ws_stream / auth	验证 IM API、WebSocket 和认证
fuxi-memory	oracle / hetu / extractor / resume	验证长期记忆和抽取行为
fuxi-orchestrator	dispatch / deliverable_handoff	验证任务派发和交付物交接
fuxi-scheduler	e2e_scheduler	验证定时触发链路
fuxi-skills	loader / staging / ledger	验证角色加载、暂存和登记
fuxi-workspace	integration	验证工作区创建与隔离

表 5.1 系统测试覆盖情况

根据 docs/status/now.md 的当前状态，cargo test --workspace --all-targets 已通过，IM PWA 前端的 pnpm test、typecheck 和 lint 也已通过。历史用户验收文档显示，抄送通路、任务树更新、对话区滚动、Resume 横幅、鼠标捕获切换等关键交互均已通过手测。

5.3 吞吐量测试

worker_n	job_sleep	tasks_n	median_w all_ms	tasks/s	理论上限	损耗
1	10ms	100	1223	81.77	100.00	18.2%
4	10ms	400	1209	330.85	400.00	17.3%
8	10ms	800	1205	663.90	800.00	17.0%
1	100ms	50	5129	9.75	10.00	2.5%
4	100ms	200	5107	39.16	40.00	2.1%
8	100ms	400	5145	77.75	80.00	2.8%
1	1000ms	10	10038	1.00	1.00	0.4%
4	1000ms	40	10030	3.99	4.00	0.3%
8	1000ms	80	10036	7.97	8.00	0.4%

表 5.2 分布式任务吞吐量测试结果

从表 5.2 可以看出，当模拟任务耗时为 10ms 时，Fuxi 在 1、4、8 个 worker 下分别达到 81.77、330.85 和 663.90 tasks/s，吞吐量随 worker 数量基本线性增长。该场景下损耗约为 17%-18%，主要来自 HTTP 往返、Tokio 调度和任务状态管理。当任务耗时增加到 100ms 或 1000ms 后，系统损耗降至 0.3%-2.8%，说明对于真实 AI Agent 场景中较重的模型推理和工具执行任务，通信调度开销占比较低。

5.4 延迟测试

metric	sample_n	min	p50	p99	max
task_dispatch	500	11.98ms	32.78ms	35.81ms	36.65ms
event_flow	500	0.04ms	0.09ms	0.25ms	1.07ms

表 5.3 任务派发与事件流延迟测试结果

表 5.3 显示，任务派发 p50 延迟为 32.78ms，p99 延迟为 35.81ms；跨节点事件流 p50 延迟仅为 0.09ms，p99 延迟为 0.25ms。事件流延迟远低于任务派发延迟，说明事件总线本身具备较好的实时性，系统交互延迟主要来自任务入队、HTTP 调度和 worker 执行链路。

Fuxi 分布式任务吞吐与事件延迟测试结果

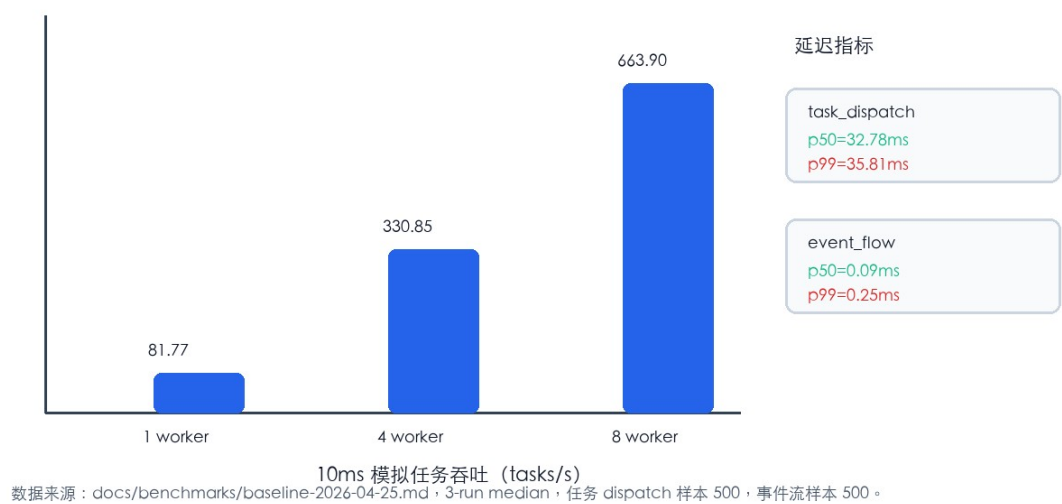


图 5.1 Fuxi 吞吐与延迟测试结果

5.5 功能验收分析

M2 验收测试重点覆盖了三个曾经影响正确性的场景：busy 状态下用户消息不丢、Codex 门客能起能派、玄女不再通过 fuxi status 反复轮询。用户验收测试则覆盖了抄送通路、门客派活后任务树迁移、对话区滚动、消息视觉、Resume 横幅和鼠标捕获切换等交互问题。这些测试说明，Fuxi 的通信链路不仅需要底

层性能，也需要在用户界面上准确表达状态，否则用户无法信任系统正在工作。

5.6 本章小结

本章通过测试覆盖表、吞吐量表、延迟表和验收记录分析了 Fuxi 的系统表现。结果表明，Fuxi 在多 worker 并行场景中具有较好的吞吐扩展性，事件流延迟达到亚毫秒级，能够满足本地 AI Agent 分布式通信系统的实时观测需求。

总结与展望

6.1 总结

本文围绕“基于 AI Agent 的高性能分布式通讯系统”完成了 Fuxi 的设计与实现。系统以 AI Agent 多任务协作为应用背景，以事件驱动通信为核心思想，构建了从协议、事件总线、编排调度、执行适配、工作区隔离、观测接口到测试验证的完整工程闭环。相比仅关注提示词和角色设定的多 Agent 框架，Fuxi 更强调本地优先、实时推送、事件持久化和执行隔离，使 Agent 协作过程具备更好的可追溯性和可恢复性。

从实现结果看，Fuxi 已支持 Claude Code 与 Codex 等 CLI Agent 的适配，其中 Claude Code 适配器支持持续会话与追加式介入，Codex 适配器采用一次 dispatch 对应一次 codex exec 进程的 one-shot 模式；系统支持 IM/PWA、Firehose、TUI 等多种观测入口，支持长期记忆、定时触发、交付物登记和跨节点任务调度。测试结果表明，系统在 8 worker、10ms 模拟任务条件下达到 663.90 tasks/s，跨节点事件流 p99 延迟为 0.25ms，验证了事件驱动架构在高性能 Agent 通信场景中的可行性。

6.2 不足与展望

当前系统仍存在一些不足。第一，分布式层目前更偏任务入队和事件回传，尚未实现多节点状态的强一致复制。第二，自动记忆抽取默认关闭，长期运行时仍需要更精细的事实筛选、冲突消解和隐私控制。第三，Agent adapter 仍主要围绕 CLI 进程，未来需要支持更多运行时形态。第四，TUI 与 IM 的交互仍可继续收敛，减少信息密度并提高任务状态表达。

后续工作可以从四个方向展开：其一，引入更完整的分布式一致性和故障转移机制；其二，完善 Agent 协议兼容性测试，使 Fuxi 与 A2A、MCP 等生态更好互操作；其三，构建更系统的安全模型，对工具调用、工作区权限和跨节点认证进行细粒度控制；其四，进一步优化 Agent 体验界面，将事件流、对话流和交付物流统一为更自然的工程协作视图。

参考文献

- [1] Wooldridge M. An Introduction to MultiAgent Systems[M]. 2nd ed. Chichester: John Wiley & Sons, 2009.
- [2] Wang L, Ma C, Feng X, et al. A Survey on Large Language Model based Autonomous Agents[EB/OL]. arXiv:2308.11432, 2023.
- [3] Li G, Hammoud H A K, Itani H, et al. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society[EB/OL]. arXiv:2303.17760, 2023.
- [4] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. arXiv:2308.08155, 2023.
- [5] Hong S, Zhuge M, Chen J, et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework[EB/OL]. arXiv:2308.00352, 2023.
- [6] Qian C, Cong X, Liu W, et al. ChatDev: Communicative Agents for Software Development[EB/OL]. arXiv:2307.07924, 2023.
- [7] Mei K, Zhu X, Xu W, et al. AIOS: LLM Agent Operating System[EB/OL]. arXiv:2403.16971, 2024.
- [8] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[EB/OL]. arXiv:2405.15793, 2024.

- [9] Wang X, Li B, Song Y, et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents[EB/OL]. arXiv:2407.16741, 2024.
- [10] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. arXiv:2210.03629, 2022.
- [11] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[EB/OL]. arXiv:2302.04761, 2023.
- [12] Karpas E, Abend O, Belinkov Y, et al. MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning[EB/OL]. arXiv:2205.00445, 2022.
- [13] Yang Y, Chai H, Song Y, et al. A Survey of AI Agent Protocols[EB/OL]. arXiv:2504.16736, 2025.
- [14] Dobrovolskyi I. Empirical Comparison of Agent Communication Protocols for Task Orchestration[EB/OL]. arXiv:2603.22823, 2026.
- [15] Agent2Agent Project. Agent2Agent Protocol Specification[EB/OL]. <https://github.com/a2aproject/A2A>, 2025.
- [16] Anthropic. Model Context Protocol Documentation[EB/OL]. <https://docs.anthropic.com/en/docs/mcp>, 2024.
- [17] Dean J, Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters[C]. OSDI, 2004: 137-150.
- [18] Ghemawat S, Gobioff H, Leung S T. The Google File System[C]. SOSP, 2003: 20-43.
- [19] Kreps J, Narkhede N, Rao J. Kafka: a Distributed Messaging System for Log Processing[C]. NetDB, 2011.
- [20] Ongaro D, Ousterhout J. In Search of an Understandable Consensus Algorithm[C]. USENIX ATC, 2014: 305-319.
- [21] SQLite. Write-Ahead Logging[EB/OL]. <https://www.sqlite.org/wal.html>.
- [22] Tokio. Channels and tokio::sync::broadcast Documentation[EB/OL]. <https://tokio.rs/tokio/tutorial/channels>.
- [23] Axum. axum::response::sse and WebSocket Documentation[EB/OL]. <https://docs.rs/axum>.
- [24] Git. git-worktree Documentation[EB/OL]. <https://git-scm.com/docs/git-worktree>.
- [25] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[EB/OL]. arXiv:2305.16291, 2023.
- [26] Li Y, et al. A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures[EB/OL]. arXiv:2506.19676, 2025.
- [27] The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption[EB/OL]. arXiv:2601.13671, 2026.
- [28] Lamport L. Time, Clocks, and the Ordering of Events in a Distributed System[J]. Communications of the ACM, 1978, 21(7): 558-565.

致谢

本文的完成离不开指导教师在选题、系统设计和论文写作方面的帮助。感谢开源社区在 Rust、AI Agent、分布式系统和文档工具方面提供的高质量资料，也感谢 Fuxi 项目开发过程中形成的测试文档、架构决策和工程日志，它们为本文提供了真实、可验证的系统材料。